Homework 1

Ball Trajectory Prediction

Multimedia Analysis in Sports Technology

March 15, 2026

Quick Start

- Build and compare **3 models**: trajectory-only, vision-only, and fusion.
- For each game, use **"Clip1" as testing**; all other clips are for training / validation.
- Prediction setting: **input frames 0–9, predict 10–19**; then **20–29, predict 30–39**, and so on.
- Ignore remaining frames if fewer than 20.
- For each game, report **ADE** and **FDE** on the **testing clip ("Clip1")**. For each testing clip, compute the metric for every prediction window and report the average across all testing windows in that clip.
- Submit `hw1.py` and a report in **CVPR 2026 format**.

Mission Overview

In modern sports analytics systems, predicting the future trajectory of the ball is an important problem. It is useful for understanding game dynamics, analyzing player actions, and supporting systems such as tennis Hawk-Eye. In this homework, you will study **future ball trajectory prediction** from different modalities and compare their effectiveness.

You must implement:

1. **Trajectory-only model**
2. **Vision-only model**
3. **Multi-modal model** (trajectory + vision)

Core Task

Observe **10 frames** and predict the ball positions in the **next 10 frames**.

Example Prediction

Figure 1 shows an example of the prediction task. The model observes the first part of the trajectory and predicts the future trajectory of the ball. In the visualization, the observed trajectory, ground-truth future trajectory, and predicted future trajectory are shown with different colors.

Example prediction windows:

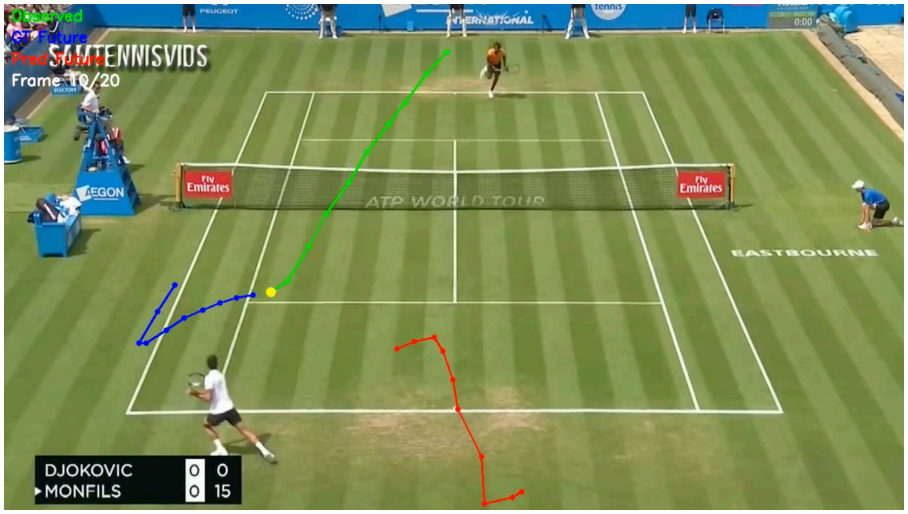

Figure 1: Example of ball trajectory prediction. The model observes the first 10 frames and predicts the ball positions in the next 10 frames. A typical visualization may show the observed trajectory in green, the ground-truth future trajectory in blue, and the predicted future trajectory in red.

Observed Frames	$\rightarrow$	**Predicted Frames**
0–9	$\rightarrow$	10–19
20–29	$\rightarrow$	30–39
40–49	$\rightarrow$	50–59

Dataset

The dataset used in this assignment is derived from the dataset introduced in the work *TrackNet* [1]. In this homework, we simplify the original tracking problem and focus on predicting future ball trajectories using the provided annotations. Download the dataset from:

https://drive.google.com/drive/folders/11rORUaQHX7I3ANkaYG4jOxXK1OYoO1Ut

The dataset contains:

- **10 games**
- each game contains several clips
- each clip contains image frames and a CSV label file

A typical structure is:

```
Dataset/
    Game1/
        Clip1/
            frames/
                0000.jpg
                0001.jpg
```

```
            ...
        label.csv
    Clip2/
        frames/
        label.csv
Game2/
    Clip1/
        frames/
            0000.jpg
            0001.jpg
            ...
        label.csv
    Clip2/
        frames/
        label.csv
```

Train / Test Split

Important Rule

For each game:

- **Clip1** is the **testing clip**
- All other clips (eg. Clip2, Clip3, ...) are for **training / validation**

Do **not** use Clip1 during training or validation.

Label Format

Each clip provides a CSV file like the following:

file name	visibility	x-coordinate	y-coordinate	status
0000.jpg	1	601	406	0
0001.jpg	1	599	386	0
0002.jpg	1	597	369	0

Meaning of each column:

- **file name**: image frame filename
- **visibility**: whether the ball is visible
- **x-coordinate**: horizontal position in pixels
- **y-coordinate**: vertical position in pixels

Invisible Ball Handling

Sometimes the ball is invisible, so the coordinates may be missing.

Example:

frame	visibility	x	y
0005.jpg	1	580	320
0006.jpg	0	None	None
0007.jpg	1	575	300

Important Rule

If the ball is invisible, that frame should be **skipped when computing ADE and FDE**. For those ball in input frame could be skipped or further preprocessed depend on your implementation.

Task Definition

Your model must use the first half of a trajectory window to predict the second half.

Example Window (Clip with 524 Frames)

Suppose a testing clip contains **524 frames**. The trajectory will be divided into non-overlapping windows of **20 frames**.

Each window follows the rule:

Use frames 0–9 as input to predict frames 10–19

This pattern then continues along the entire clip.

Input Frames	$\rightarrow$	**Prediction Target**
0 – 9	$\rightarrow$	10 – 19
20 – 29	$\rightarrow$	30 – 39
40 – 49	$\rightarrow$	50 – 59
$\vdots$	$\rightarrow$	$\vdots$
500 – 509	$\rightarrow$	510 – 519

Since the clip contains **524 frames**, the remaining frames:

520 – 523

do not form a complete 20-frame window and should therefore be **ignored**.

Recommended Training Setting

We recommend training your model using the same rule:

10 input frames $\rightarrow$ predict 10 future frames

You may train in another way, but evaluation will follow the window protocol above.

Models to Implement

1. Trajectory-only Model

Input:

$$(x_0, y_0), (x_1, y_1), \ldots, (x_9, y_9)$$

Output:

$$(x_{10}, y_{10}), (x_{11}, y_{11}), \ldots, (x_{19}, y_{19})$$

Possible approaches include:

- linear regression
- polynomial fitting
- MLP
- RNN / LSTM

2. Vision-only Model

Input:

$$\text{10 image frames}$$

Output:

$$(x_{10}, y_{10}), (x_{11}, y_{11}), \ldots, (x_{19}, y_{19})$$

Possible approaches include:

- CNN feature extraction
- CNN + MLP
- CNN + RNN / LSTM

3. Fusion Model

Input:

- 10 trajectory coordinates $(x_0, y_0), (x_1, y_1), \ldots, (x_9, y_9)$
- 10 image frames

Output:

$$(x_{10}, y_{10}), (x_{11}, y_{11}), \ldots, (x_{19}, y_{19})$$

This part investigates whether and how multimodal fusion can improve prediction performance, e.g., via feature concatenation or cross-attention mechanisms.

Evaluation

You must report results on the **testing clip "Clip1" of each of the 10 games**. For each testing clip, first compute ADE and FDE for every valid prediction window (e.g., 0–9 → 10–19, 20–29 → 30–39, etc.), and then report the **average ADE** and **average FDE** across all testing windows in

that clip. In other words, each game should have one final ADE and one final FDE, obtained by averaging over all valid testing windows in its Clip1.

Average Displacement Error (ADE)

ADE measures the average Euclidean distance between predicted and ground-truth coordinates, in **pixels (px)**:

$$ADE = \frac{1}{T}\sum_{t=1}^{T}\|\hat{p}_t - p_t\|$$

Final Displacement Error (FDE)

FDE measures the Euclidean distance between the predicted and ground-truth final point, in **pixels (px)**:

$$FDE = \|\hat{p}_T - p_T\|$$

For a testing clip containing N valid prediction windows, the reported clip-level metrics are:

$$ADE_{\text{clip}} = \frac{1}{N}\sum_{i=1}^{N} ADE_i$$

$$FDE_{\text{clip}} = \frac{1}{N}\sum_{i=1}^{N} FDE_i$$

where ADE_i and FDE_i are computed for the i-th prediction window.

Example

Suppose the prediction errors of 4 future frames are measured in pixels (px):

$$5,\ 7,\ 8,\ 9$$

Then:

$$ADE = \frac{5+7+8+9}{4} = 7.25$$

$$FDE = 9$$

Important Rule

When computing ADE / FDE:

- skip predicted frames whose ground-truth ball is invisible
- compute ADE and FDE for each valid prediction window
- average the window-level metrics to obtain the final ADE and FDE for that testing clip

Grading

Component	Score
Trajectory-only implementation	20%
Vision-only implementation	20%
Fusion implementation	20%
Final report	40%

Report Format

Your report must use the **CVPR 2026 submission template**:

https://github.com/cvpr-org/author-kit/archive/refs/tags/CVPR2026-v1(latex).zip

Required Report Structure

Your report should include:

1. **Abstract** Briefly summarize your approach, results, and findings.
2. **Introduction** Describe the problem and its relevance. Introduce trajectory prediction and possible models such as CNN, RNN, and LSTM.
3. **Methodology** Explain your model design, fusion strategy, and training method. Include a model diagram if possible.
4. **Experiments and Discussion** Include:
   - experimental settings (batch size, epochs, learning rate, optimizer)
   - data preprocessing and augmentation
   - ADE / FDE results for each of the 10 games separately
   - For each game, ADE and FDE are computed for every prediction window in the testing clip, and the final reported metric is the average across all valid windows.

	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	Avg
Trajectory ADE											
Trajectory FDE											
Vision ADE											
Vision FDE											
Fusion ADE											
Fusion FDE											

   - loss curves
   - ablation studies
   - analysis of challenges, limitations, and observations
   - examples of predicted trajectories
5. **Conclusion** Summarize key findings and suggest future improvements.

Submission Format

Submission Checklist

Submit a single ZIP file named:

`YourStudentID_Name.zip`

Example:

`NE6134079_ 劉世文.zip`

The ZIP file should contain:

```
YourStudentID_Name/
    hw1.py
    YourStudentID_Name.pdf
```

- `hw1.py`: clean and well-commented Python script
- `YourStudentID_Name.pdf`: report in CVPR 2026 format

Final Reminder

Plagiarism Warning

Do **not** copy code or reports from other students or external sources.
Any plagiarism will result in **0 points**.

If you encounter any difficulties, please contact the TA via email: ne6134079@gs.ncku.edu.tw

References

[1] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. Ik, and W.-C. Peng, "TrackNet: A Deep Learning Network for Tracking High-speed and Tiny Objects in Sports Applications," in *Proc. 16th IEEE Int. Conf. on Advanced Video and Signal Based Surveillance (AVSS)*, 2019, pp. 1–8.